

Transport Layer

The transport layer is the layer responsible for process-to-process communication between hosts. It sits above the network layer and below the application layer.

In the TCP/IP model:

```
Application Layer -> HTTP, DNS, SMTP, FTP, SSH
Transport Layer   -> TCP, UDP
Internet Layer    -> IP, ICMP
Network Access    -> Ethernet, Wi-Fi
```

In the OSI model, the transport layer is Layer 4.

Core Responsibilities

The transport layer provides services such as:

- Process-to-process delivery
- Segmentation and reassembly
- Multiplexing and demultiplexing
- Reliability, if the protocol supports it
- Error detection
- Flow control
- Congestion control
- Connection establishment and termination, if connection-oriented
- Port-based communication

Host-To-Host vs Process-To-Process

The network layer provides host-to-host delivery.

Example:

```
192.168.1.10 -> 142.250.183.14
```

The transport layer provides process-to-process delivery.

Example:

```
192.168.1.10:51524 -> 142.250.183.14:443
```

Here:

- 192.168.1.10 is the client IP address.
- 51524 is the client-side ephemeral port.
- 142.250.183.14 is the server IP address.
- 443 is the server port for HTTPS.

Transport Layer Protocols

The two most important transport layer protocols are:

- TCP, Transmission Control Protocol
- UDP, User Datagram Protocol

Other transport-layer or transport-like protocols include:

- SCTP, Stream Control Transmission Protocol
- DCCP, Datagram Congestion Control Protocol
- QUIC, built over UDP and used by HTTP/3

For placements, TCP and UDP are the most important.

TCP

TCP stands for Transmission Control Protocol.

TCP is:

- Connection-oriented
- Reliable
- Ordered
- Byte-stream based
- Full-duplex
- Flow-controlled
- Congestion-controlled

TCP is used when correctness matters more than raw speed.

Common TCP applications:

- HTTP/HTTPS
- FTP
- SMTP
- IMAP/POP3
- SSH
- Telnet
- Database connections

TCP Features

1. Connection-Oriented

Before sending actual data, TCP creates a logical connection between sender and receiver using the 3-way handshake.

2. Reliable Delivery

TCP ensures that data reaches the receiver correctly. It uses:

- Sequence numbers
- Acknowledgements
- Retransmission
- Checksums
- Timers

3. Ordered Delivery

TCP delivers bytes to the application in the same order in which they were sent.

If segments arrive out of order, TCP buffers them until missing data arrives.

4. Byte Stream

TCP does not preserve message boundaries.

If an application sends:

```
send("Hello")  
send("World")
```

The receiver may receive:

```
"HelloWorld"
```

or:

```
"Hel"  
"loWor"  
"ld"
```

TCP sees data as a continuous stream of bytes, not separate messages.

5. Full-Duplex Communication

Both sides can send and receive data simultaneously.

6. Flow Control

TCP prevents a fast sender from overwhelming a slow receiver.

7. Congestion Control

TCP prevents the sender from overwhelming the network.

UDP

UDP stands for User Datagram Protocol.

UDP is:

- Connectionless
- Unreliable
- Unordered
- Message-oriented
- Lightweight
- Fast

UDP is used when speed, simplicity, or real-time behavior matters more than guaranteed delivery.

Common UDP applications:

- DNS
- DHCP
- VoIP
- Online gaming
- Video streaming
- Live audio/video calls
- TFTP
- SNMP
- NTP

UDP Features

1. Connectionless

UDP does not establish a connection before sending data.

A sender can directly send a datagram to the receiver.

2. Unreliable

UDP does not guarantee:

- Delivery
- Ordering
- Duplicate protection
- Retransmission

If reliability is needed, the application must implement it.

3. Message-Oriented

UDP preserves message boundaries.

If the sender sends one UDP datagram, the receiver receives that datagram as one unit, if it arrives.

4. Low Overhead

UDP has a smaller header than TCP.

UDP header size: 8 bytes.

TCP header size: minimum 20 bytes.

5. No Built-In Flow Control

UDP does not control how fast the sender sends.

6. No Built-In Congestion Control

UDP does not automatically reduce its sending rate when the network is congested.

TCP vs UDP

Feature	TCP	UDP
Full form	Transmission Control Protocol	User Datagram Protocol
Connection	Connection-oriented	Connectionless
Reliability	Reliable	Unreliable
Ordering	Maintains order	Does not maintain order
Speed	Slower due to overhead	Faster due to low overhead
Header size	Minimum 20 bytes	8 bytes
Data type	Byte stream	Datagram/message
Flow control	Yes	No
Congestion control	Yes	No built-in congestion control
Acknowledgement	Yes	No
Retransmission	Yes	No
Broadcast/multicast	Not suitable	Suitable
Use cases	Web, file transfer, email, SSH	DNS, gaming, VoIP, streaming

When To Use TCP

Use TCP when:

- Data must be correct.
- Data must be complete.
- Data must arrive in order.
- Delay is acceptable.

Examples:

- Downloading a file
- Loading a web page
- Sending an email
- Remote login using SSH

When To Use UDP

Use UDP when:

- Low latency is important.
- Small data packets are sent.
- Some loss is acceptable.
- Application can handle reliability itself.

Examples:

- Video call
- Online multiplayer game
- DNS query
- Live streaming

TCP Header

A TCP segment contains a TCP header and data.

Important TCP header fields:

Field	Purpose
Source port	Port number of sender process
Destination port	Port number of receiver process
Sequence number	Identifies byte number of data
Acknowledgement number	Next expected byte
Header length	Size of TCP header
Flags	Control bits such as SYN, ACK, FIN, RST
Window size	Receiver's available buffer size
Checksum	Error detection

Field	Purpose
Urgent pointer	Used when urgent data exists
Options	Extra features such as MSS and window scaling

TCP Flags

Flag	Meaning
SYN	Synchronize sequence numbers; used to open connection
ACK	Acknowledgement field is valid
FIN	Finish; used to close connection gracefully
RST	Reset connection immediately
PSH	Push data to application quickly
URG	Urgent pointer is valid
ECE	Explicit Congestion Notification Echo
CWR	Congestion Window Reduced

UDP Header

UDP has a simple 8-byte header.

Field	Size	Purpose
Source port	16 bits	Sender port
Destination port	16 bits	Receiver port
Length	16 bits	Header plus data length
Checksum	16 bits	Error detection

Port Numbers

A port number identifies a process or service on a host.

IP address identifies the machine.

Port number identifies the application/process inside the machine.

Example:

```
IP address: 172.217.166.46
```

Port: 443

This means:

Connect to the HTTPS service running on host 172.217.166.46.

Port Number Range

Port numbers are 16-bit values.

Range:

0 to 65535

Types of Ports

Range	Name	Use
0-1023	Well-known ports	Standard services
1024-49151	Registered ports	Applications and services
49152-65535	Dynamic/ephemeral ports	Temporary client-side ports

Common Port Numbers

Port	Protocol	Service
20	TCP	FTP data
21	TCP	FTP control
22	TCP	SSH
23	TCP	Telnet
25	TCP	SMTP
53	TCP/UDP	DNS
67	UDP	DHCP server
68	UDP	DHCP client
69	UDP	TFTP
80	TCP	HTTP
110	TCP	POP3
123	UDP	NTP

Port	Protocol	Service
143	TCP	IMAP
161	UDP	SNMP
443	TCP/UDP	HTTPS / HTTP/3 over QUIC
465	TCP	SMTPS
587	TCP	SMTP submission
993	TCP	IMAPS
995	TCP	POP3S

Sockets

A socket is an endpoint of communication.

A socket is usually identified by:

```
IP address + port number + protocol
```

Example:

```
192.168.1.5:50000 using TCP
```

Socket Pair

A TCP connection is uniquely identified by a 4-tuple:

```
Source IP, Source Port, Destination IP, Destination Port
```

Example:

```
192.168.1.5:51524 -> 142.250.183.14:443
```

This allows a server to handle many clients on the same port.

Example:

```
Client A: 10.0.0.1:50100 -> Server: 20.0.0.1:443  
Client B: 10.0.0.2:50101 -> Server: 20.0.0.1:443
```

```
Client C: 10.0.0.3:50102 -> Server: 20.0.0.1:443
```

All clients connect to server port 443, but each connection is unique because the full 4-tuple is different.

Socket Types

Common socket types:

- Stream socket: uses TCP
- Datagram socket: uses UDP
- Raw socket: used for low-level packet access

Server Socket vs Client Socket

Server:

- Creates socket
- Binds to a known port
- Listens for connections
- Accepts client connections

Client:

- Creates socket
- Usually gets an ephemeral port
- Connects to server IP and port

TCP server flow:

```
socket() -> bind() -> listen() -> accept() -> read/write -> close()
```

TCP client flow:

```
socket() -> connect() -> read/write -> close()
```

UDP server flow:

```
socket() -> bind() -> recvfrom() -> sendto()
```

UDP client flow:

```
socket() -> sendto() -> recvfrom()
```

Multiplexing and Demultiplexing

Multiplexing

Multiplexing means collecting data from multiple application processes and passing it to the network layer.

Example:

```
Browser, email client, and chat app all send data through the transport layer.
```

The transport layer adds source and destination port numbers.

Demultiplexing

Demultiplexing means receiving segments from the network layer and delivering them to the correct application process using port numbers.

Example:

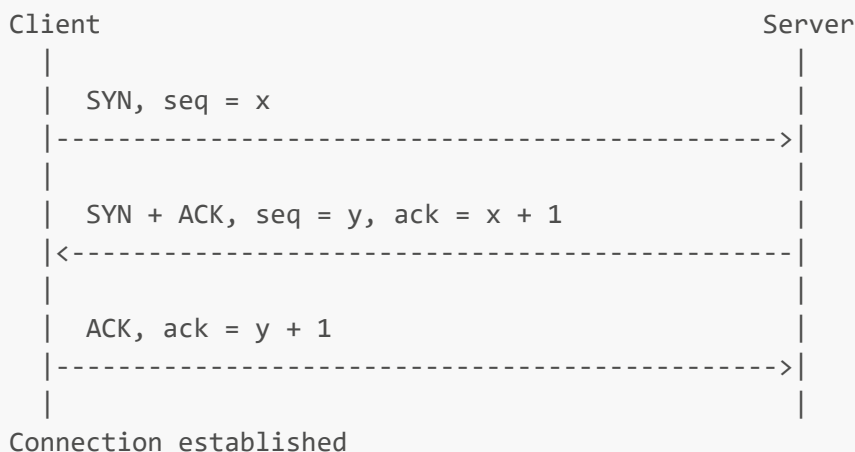
```
Packet for port 80 -> web server  
Packet for port 25 -> mail server  
Packet for port 53 -> DNS server
```

TCP 3-Way Handshake

The TCP 3-way handshake establishes a TCP connection.

It synchronizes sequence numbers between client and server.

Steps



Step 1: SYN

Client sends a SYN segment to the server.

It contains the client's initial sequence number.

```
SYN = 1  
Sequence number = x
```

Step 2: SYN-ACK

Server replies with SYN-ACK.

It acknowledges the client's SYN and sends its own initial sequence number.

```
SYN = 1  
ACK = 1  
Sequence number = y  
Acknowledgement number = x + 1
```

Step 3: ACK

Client sends final ACK.

```
ACK = 1  
Acknowledgement number = y + 1
```

After this, both sides can send data.

Why 3 Steps Are Needed

The 3-way handshake confirms:

- Client can send.
- Client can receive.
- Server can send.
- Server can receive.
- Both sides agree on initial sequence numbers.

Why Not 2-Way Handshake?

A 2-way handshake can create problems with delayed duplicate connection requests.

Example:

1. Client sends old SYN.

2. Old SYN is delayed in the network.
3. Server later receives it and believes a new connection is requested.
4. Server may allocate resources unnecessarily.

The third ACK helps confirm that the client really wants the connection.

Initial Sequence Number

TCP does not usually start sequence numbers from 0.

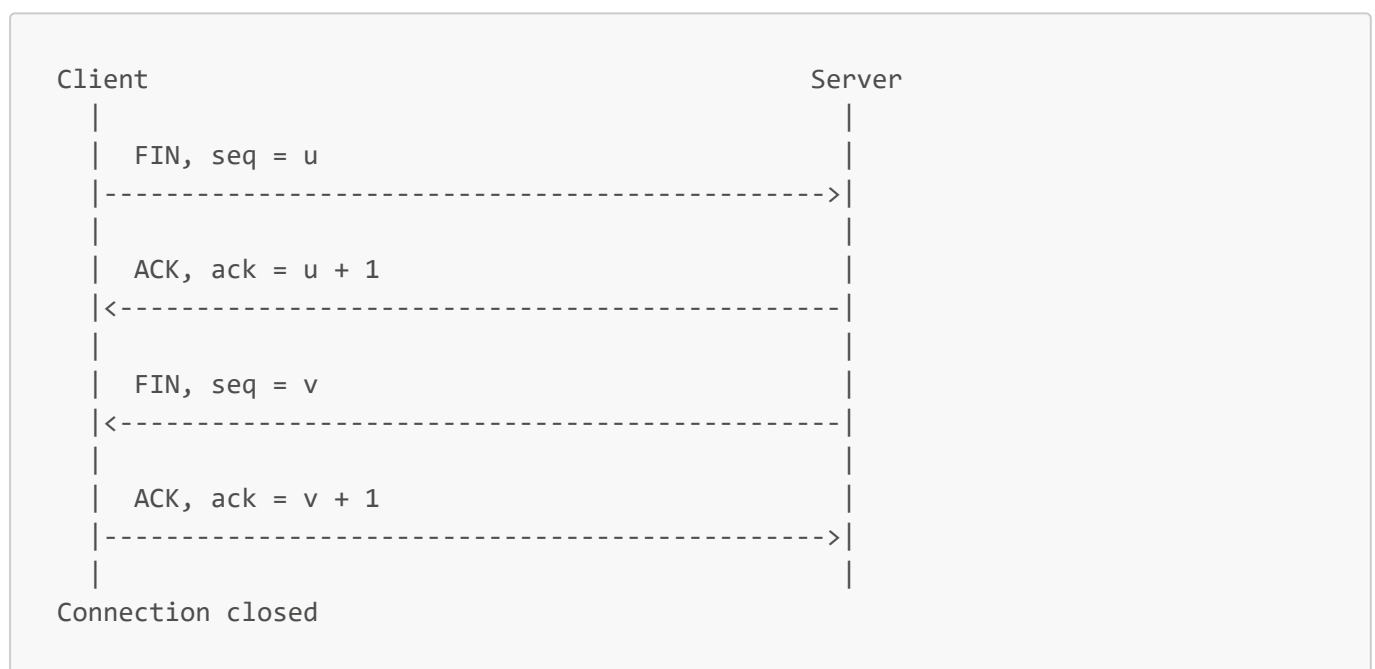
It chooses an Initial Sequence Number, or ISN, to reduce confusion with old duplicate segments.

TCP Connection Close

TCP connection termination usually uses a 4-way handshake.

TCP is full-duplex, so each direction must be closed independently.

Graceful Close



Why 4 Steps?

Because TCP is full-duplex.

When one side sends FIN, it means:

I have no more data to send.

But the other side may still have data to send.

So each direction is closed separately.

FIN

FIN means graceful close.

The sender has finished sending data.

RST

RST means reset.

It immediately aborts the connection.

RST is used when:

- A connection request reaches a closed port.
- An application wants to abort the connection.
- A serious error occurs.

Half-Close

TCP supports half-close.

One side can stop sending data while still receiving data.

Example:

```
Client sends FIN.  
Server ACKs it.  
Server can still send remaining data.
```

TCP Connection States

Important TCP states:

State	Meaning
CLOSED	No connection
LISTEN	Server waiting for connection
SYN-SENT	Client has sent SYN
SYN-RECEIVED	Server received SYN and sent SYN-ACK
ESTABLISHED	Connection active
FIN-WAIT-1	FIN sent, waiting for ACK
FIN-WAIT-2	FIN acknowledged, waiting for peer FIN
CLOSE-WAIT	Received FIN, waiting for local app to close

State	Meaning
CLOSING	Both sides sent FIN simultaneously
LAST-ACK	Waiting for ACK of final FIN
TIME-WAIT	Waiting before final close

TIME-WAIT

TIME-WAIT happens after the active closer sends the final ACK.

It exists to:

- Ensure the final ACK can be retransmitted if lost.
- Allow old duplicate segments to expire before reusing the same connection tuple.

TIME-WAIT usually lasts 2 times the Maximum Segment Lifetime.

This is often written as:

```
2MSL
```

Flow Control

Flow control prevents the sender from overwhelming the receiver.

It is about the receiver's capacity.

Example:

```
Sender is fast.  
Receiver has a small buffer.  
Without flow control, receiver buffer overflows.
```

TCP uses a sliding window mechanism for flow control.

Receiver Window

The receiver advertises how much free buffer space it has.

This advertised value is called:

```
Receiver Window, rwnd
```

The sender must not send more unacknowledged data than the receiver's advertised window.

Formula:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$$

Zero Window

If the receiver buffer becomes full, it advertises:

$$\text{rwnd} = 0$$

The sender stops sending data.

To avoid deadlock, TCP uses a persist timer and sends zero-window probes to check whether the receiver has space again.

Congestion Control

Congestion control prevents the sender from overwhelming the network.

It is about the network's capacity.

Symptoms of congestion:

- Packet loss
- Increasing delay
- Router buffer overflow
- Retransmissions
- Reduced throughput

TCP congestion control dynamically adjusts the amount of data in flight using:

Congestion Window, cwnd

Flow Control vs Congestion Control

Feature	Flow Control	Congestion Control
Protects	Receiver	Network
Based on	Receiver buffer	Network condition
Main variable	rwnd	cwnd
Problem avoided	Receiver buffer overflow	Network congestion
Controlled by	Receiver advertisement	Sender algorithm

Effective Sending Window

TCP sender is limited by both flow control and congestion control.

```
Effective window = min(rwnd, cwnd)
```

This means TCP can send only as much data as both the receiver and the network can handle.

TCP Congestion Control Phases

Classic TCP congestion control has these main phases:

- Slow start
- Congestion avoidance
- Fast retransmit
- Fast recovery

1. Slow Start

Slow start is used at the beginning of a connection or after a timeout.

Despite the name, it grows quickly.

The congestion window starts small and grows exponentially.

Common idea:

```
cwnd starts at 1 MSS  
cwnd increases by 1 MSS for each ACK
```

This roughly doubles cwnd every RTT.

Example:

```
RTT 1: cwnd = 1 MSS  
RTT 2: cwnd = 2 MSS  
RTT 3: cwnd = 4 MSS  
RTT 4: cwnd = 8 MSS  
RTT 5: cwnd = 16 MSS
```

2. Slow Start Threshold

The slow start threshold is called:

```
ssthresh
```

When:

```
cwnd >= ssthresh
```

TCP moves from slow start to congestion avoidance.

3. Congestion Avoidance

In congestion avoidance, cwnd grows linearly.

Typical behavior:

```
cwnd increases by about 1 MSS per RTT
```

This avoids increasing the sending rate too aggressively.

4. Timeout

A timeout is treated as a strong sign of congestion.

Common TCP reaction:

```
ssthresh = cwnd / 2  
cwnd = 1 MSS  
return to slow start
```

5. Duplicate ACKs

If a segment is lost, the receiver may keep acknowledging the last in-order byte.

Repeated ACKs with the same acknowledgement number are called duplicate ACKs.

6. Fast Retransmit

If sender receives 3 duplicate ACKs, it assumes a segment was lost and retransmits it immediately, without waiting for timeout.

```
3 duplicate ACKs -> fast retransmit
```

7. Fast Recovery

After fast retransmit, TCP does not always go back to $\text{cwnd} = 1 \text{ MSS}$.

Instead, it reduces cwnd and continues more carefully.

Typical behavior:

```
ssthresh = cwnd / 2
cwnd = ssthresh
```

This is less drastic than timeout recovery.

TCP Tahoe vs Reno

Feature	TCP Tahoe	TCP Reno
Slow start	Yes	Yes
Congestion avoidance	Yes	Yes
Fast retransmit	Yes	Yes
Fast recovery	No	Yes
After 3 duplicate ACKs	$\text{cwnd} = 1 \text{ MSS}$	cwnd reduced, fast recovery

Sliding Window

Sliding window is a technique that allows multiple packets or bytes to be sent before waiting for acknowledgements.

It improves efficiency compared to stop-and-wait.

Stop-And-Wait Problem

In stop-and-wait:

```
Send packet 1
Wait for ACK 1
Send packet 2
Wait for ACK 2
```

This wastes bandwidth when RTT is high.

Sliding Window Idea

The sender can send several segments before receiving ACKs.

Example with window size 4:

```
Send 1, 2, 3, 4
Receive ACK 1
Window slides
Send 5
```

Sender Window

The sender window contains data that:

- Has been sent but not yet acknowledged
- Can be sent immediately

Conceptually:

```
[ACKed data][Sent but unACKed][Can send now][Cannot send yet]
```

Receiver Window

The receiver window represents how much data the receiver can accept.

It depends on available buffer space.

TCP Sliding Window

TCP uses byte-based sliding window, not packet-based sliding window.

This means sequence numbers identify bytes, not segments.

Example:

If a TCP segment carries 1000 bytes and starts with sequence number 5000:

```
Bytes: 5000 to 5999
Next expected byte: 6000
ACK number: 6000
```

Cumulative ACK

TCP commonly uses cumulative acknowledgements.

ACK number means:

```
I have received all bytes before this number, and I expect this byte next.
```

Example:

```
ACK = 7000
```

Meaning:

```
All bytes up to 6999 have been received.  
Next expected byte is 7000.
```

Selective Acknowledgement

Selective Acknowledgement, or SACK, lets the receiver tell the sender about non-contiguous blocks of data received.

This helps TCP retransmit only missing data instead of retransmitting many segments.

Reliable Data Transfer in TCP

TCP reliability is achieved using:

- Sequence numbers
- Acknowledgements
- Retransmission timers
- Checksums
- Duplicate ACK detection

Sequence Numbers

TCP sequence numbers count bytes.

If sequence number is 1000 and the segment has 500 bytes, the next sequence number is:

```
1500
```

Acknowledgement Numbers

Acknowledgement number is the next byte expected.

If receiver gets bytes 1000 to 1499, it sends:

```
ACK = 1500
```

Retransmission

TCP retransmits data when:

- Retransmission timeout occurs
- 3 duplicate ACKs are received

Checksum

TCP checksum detects corruption in TCP header and data.

If checksum fails, the segment is discarded.

MSS, MTU, and Fragmentation

MTU

MTU stands for Maximum Transmission Unit.

It is the largest frame payload a link-layer network can carry.

Ethernet MTU is commonly:

1500 bytes

MSS

MSS stands for Maximum Segment Size.

It is the maximum amount of TCP data in a segment.

For IPv4 over Ethernet:

MTU = 1500 bytes
IP header = 20 bytes
TCP header = 20 bytes
MSS = 1460 bytes

Formula:

$$\text{MSS} = \text{MTU} - \text{IP header} - \text{TCP header}$$

Fragmentation

IP fragmentation happens when a packet is larger than the MTU and must be split.

TCP tries to avoid fragmentation by using MSS.

Important TCP Timers

Retransmission Timer

Used to retransmit data if ACK does not arrive in time.

Persist Timer

Used when receiver advertises zero window.

The sender periodically checks if the receiver window has opened.

Keepalive Timer

Used to check whether an idle connection is still alive.

TIME-WAIT Timer

Keeps connection information for 2MSL after closing.

TCP Problems and Solutions

Silly Window Syndrome

Silly Window Syndrome happens when very small amounts of data are sent repeatedly, causing inefficient use of bandwidth.

Solutions:

- Nagle's algorithm on sender side
- Clark's solution on receiver side

Nagle's Algorithm

Nagle's algorithm reduces tiny packet transmission.

Basic idea:

If there is unacknowledged data, buffer small outgoing data until ACK arrives or enough data accumulates.

Good for:

- Reducing overhead
- Improving network efficiency

Bad for:

- Some low-latency applications

Delayed ACK

Delayed ACK means receiver waits briefly before sending ACK, hoping it can combine ACK with outgoing data.

This reduces ACK traffic but can sometimes increase latency.

Important Interview Distinctions

TCP Segment vs UDP Datagram

TCP unit of data:

Segment

UDP unit of data:

Datagram

TCP Connection vs UDP Communication

TCP has connection state.

UDP does not maintain connection state at transport layer.

Reliable vs Unreliable

Reliable means the protocol detects loss and retransmits.

Unreliable means the protocol does not guarantee delivery.

Unreliable does not mean useless. UDP is excellent for real-time use cases.

Flow Control vs Error Control

Flow control:

Controls sender speed so receiver is not overwhelmed.

Error control:

Detects and handles lost, damaged, or duplicate data.

Common Placement Questions

1. What is the transport layer?

The transport layer provides process-to-process communication between applications running on different hosts. It uses port numbers to identify processes and provides services such as reliability, flow control, congestion control, multiplexing, and demultiplexing.

2. What is the difference between TCP and UDP?

TCP is connection-oriented, reliable, ordered, and provides flow and congestion control. UDP is connectionless, unreliable, unordered, lightweight, and faster. TCP is used for web, email, file transfer, and SSH. UDP is used for DNS, VoIP, gaming, streaming, and real-time applications.

3. Why is TCP reliable?

TCP is reliable because it uses sequence numbers, acknowledgements, retransmissions, checksums, timers, and ordered delivery.

4. What is the 3-way handshake?

The 3-way handshake is TCP's connection establishment process:

```
Client -> Server: SYN
Server -> Client: SYN-ACK
Client -> Server: ACK
```

It synchronizes sequence numbers and confirms that both sides can send and receive.

5. Why does TCP use a 3-way handshake and not 2-way?

A 3-way handshake protects against old duplicate SYN packets and confirms bidirectional communication. A 2-way handshake may cause half-open or false connections due to delayed duplicate packets.

6. How is a TCP connection closed?

TCP usually closes using a 4-way handshake:

```
FIN -> ACK -> FIN -> ACK
```

Because TCP is full-duplex, each direction is closed separately.

7. What is TIME-WAIT?

TIME-WAIT is the state where the active closer waits before fully closing the connection. It ensures the final ACK can be retransmitted if needed and old duplicate packets expire.

8. What is flow control?

Flow control prevents a fast sender from overwhelming a slow receiver. TCP uses the receiver window, or `rwnd`, to tell the sender how much data the receiver can accept.

9. What is congestion control?

Congestion control prevents too much data from being injected into the network. TCP uses congestion window, or `cwnd`, and algorithms such as slow start, congestion avoidance, fast retransmit, and fast recovery.

10. Difference between flow control and congestion control?

Flow control protects the receiver. Congestion control protects the network. Flow control uses `rwnd`. Congestion control uses `cwnd`.

11. What is sliding window?

Sliding window allows multiple bytes or packets to be sent before waiting for ACKs. It improves throughput by keeping the network busy.

12. What is a port number?

A port number identifies a process or service on a host. IP identifies the machine, and port identifies the application.

13. What is a socket?

A socket is an endpoint of communication. It is identified by IP address, port number, and protocol.

14. Can two applications use the same port?

Usually, two applications cannot bind to the exact same IP, port, and protocol combination at the same time. However, a TCP server can handle many clients on the same listening port because each connection has a unique 4-tuple.

15. What is an ephemeral port?

An ephemeral port is a temporary port assigned to a client for a connection. It usually comes from the high port range.

16. Why is UDP used for DNS?

DNS queries are usually small and need low latency. UDP avoids connection setup overhead. If a DNS response is too large or reliability is needed, DNS can use TCP.

17. Why is UDP used in video calls?

Video calls prefer low latency. If one packet is lost, waiting for retransmission may be worse than skipping it. UDP allows real-time applications to handle loss according to their needs.

18. Does UDP have checksum?

Yes. UDP has a checksum field for error detection. In IPv4, UDP checksum is optional, but in IPv6 it is mandatory.

19. Is TCP always slower than UDP?

Not always, but TCP has more overhead because of connection setup, acknowledgements, reliability, ordering, flow control, and congestion control. UDP is simpler and often has lower latency.

20. What is the difference between cwnd and rwnd?

cwnd is the congestion window maintained by the sender based on network congestion.

rwnd is the receiver window advertised by the receiver based on buffer space.

Actual sending limit:

```
min(cwnd, rwnd)
```

Quick Revision Sheet

TCP

- Connection-oriented
- Reliable
- Ordered
- Byte-stream protocol
- Uses 3-way handshake
- Uses 4-way close
- Has flow control
- Has congestion control
- Uses sequence numbers and ACKs

UDP

- Connectionless
- Unreliable
- Unordered
- Message-oriented protocol
- No handshake
- No built-in retransmission

- No built-in flow control
- No built-in congestion control
- Small 8-byte header

3-Way Handshake

SYN
SYN-ACK
ACK

4-Way Close

FIN
ACK
FIN
ACK

Key Formulas

Effective TCP window = $\min(\text{rwnd}, \text{cwnd})$
 $\text{MSS} = \text{MTU} - \text{IP header} - \text{TCP header}$

Key Terms

Term	Meaning
Port	Identifies application process
Socket	Communication endpoint
Segment	TCP data unit
Datagram	UDP data unit
rwnd	Receiver window
cwnd	Congestion window
MSS	Maximum TCP data per segment
MTU	Maximum packet size supported by link
RTT	Round-trip time
ACK	Acknowledgement
SYN	Starts TCP connection

Term	Meaning
FIN	Graceful close
RST	Abrupt reset

One-Minute Interview Answer

The transport layer provides process-to-process communication using port numbers. TCP and UDP are the main transport protocols. TCP is connection-oriented, reliable, ordered, and uses sequence numbers, ACKs, retransmissions, flow control, congestion control, a 3-way handshake for connection setup, and usually a 4-way handshake for connection termination. UDP is connectionless, lightweight, faster, and does not guarantee delivery or ordering, so it is used for DNS, streaming, gaming, and real-time applications. TCP controls receiver overload using `rwnd` and network congestion using `cwnd`, with the actual sending window being the minimum of both.